Job Application Agent

Codebase Technical Assessment

An agentic AI system that scores professional profiles against job descriptions using LangGraph, LlamaIndex RAG, and multi-provider LLMs

Aspect	Summary
Domain	Career / recruitment automation (personal ATS-style scorer)
Core pattern	Agentic pipeline (LangGraph) + Retrieval-Augmented Generation
Primary language	Python 3.13
Interfaces	Streamlit web app + CLI (main.py)
LLMs	Groq Llama-3.3-70B (primary) · Azure OpenAI GPT-4o-mini (fallback)
Retrieval	LlamaIndex + Chroma + local HuggingFace embeddings
Job data	Exa neural search / contents API (SDK + optional MCP)

Prepared for technical assessment. All statements below are grounded in the actual repository modules: `app.py`, `agents/graph.py`, `agents/nodes.py`, `tools/vector_store.py`, `tools/exa_mcp.py`, `report.py` and `state.py`.

1 Business Use Case

The problem

Tailoring a CV or professional profile to each individual job posting is slow, repetitive and subjective. Applicant Tracking Systems (ATS) reward keyword and experience alignment, but a candidate cannot easily tell — for any given job — whether they should apply as an **individual contributor (IC)** or in a **leadership** capacity, which specific requirements they miss, and what to change to raise their match score.

The solution

The **Job Application Agent** is an AI assistant that ingests a job description, extracts its structured requirements, and scores the candidate's two maintained profiles (IC and Leadership) against it. For each job it returns an ATS-style score out of 100 per profile, recommends the stronger profile, lists strengths and gaps, and proposes grounded, actionable “levers” to close those gaps. Every run is appended to a persistent log and a cumulative PDF ATS report is rebuilt automatically.

Who uses it and how

- **User:** a single job seeker evaluating many postings quickly and objectively.
- **Inputs (3 reliable modes):** a single **Job URL**, a **file/paste of many Job URLs** (bulk scoring), or **pasted JD text**.
- **Outputs:** IC score /100, Leadership score /100, recommended profile, strengths, gaps, improvement levers, and a downloadable cumulative ATS report PDF.
- **Value:** converts a subjective, manual triage into a repeatable, evidence-grounded scoring workflow — decide fast where to invest application effort.

Business outcomes

- Faster shortlisting of which roles are worth applying to.
- Objective, side-by-side IC-vs-Leadership positioning per role.
- A running record (JSON + PDF) that documents fit across all evaluated jobs.
- Zero external data/embedding cost — the corpus and vectors stay fully local.

2 Solution Architecture

The system is orchestrated as a **LangGraph state machine** — a typed, linear pipeline of five nodes. Each node reads and writes a shared, typed state object (`JobAgentState`) and emits progress messages that the Streamlit UI streams live.

`Pipeline:` `START → fetch_jd → parse_jd → score_profiles → analyse_gaps → update_ats → END`

Stage	Node	What it does	Key tech
1	fetch_jd	Retrieves the JD text from a URL (or search query) and captures the source URL.	Exa (exa-py)
2	parse_jd	Extracts structured fields (title, company, requirements, nice-to-haves, domain, seniority) into a Pydantic model.	LlamaIndex structured_predict
3	score_profiles	RAG: retrieves the most relevant IC & Leadership profile chunks and scores both against the JD requirements.	Chroma + LlamaIndex retriever + Groq LLM

Stage	Node	What it does	Key tech
4	analyse_gaps	Turns each identified gap into a concrete, grounded improvement “lever”.	Groq LLM
5	update_ats	Appends the run to ats_log.json and rebuilds the cumulative ATS-Report.pdf.	ReportLab

Shared state

A single `TypedDict` carries data across nodes: the job URL / search query, the raw and parsed JD, IC & Leadership scores, best profile, strengths, gaps, levers, the ATS entry, and a LangGraph-reduced message log. This makes the flow inspectable and each node independently testable.

Two front-ends

- **Streamlit** (`app.py`): a two-tab UI — an **Agent** tab for scoring and a **Reports** tab for the cumulative ATS report.
- **CLI** (`main.py`): headless single-run scoring plus a `--refresh-profiles` re-index command.

3 Tech Stack, Frameworks & Tools

The solution deliberately combines two AI frameworks — **LangChain/LangGraph** for orchestration and LLM provider abstraction, and **LlamaIndex** for structured extraction and RAG ingestion/retrieval — each used where it is strongest.

Layer	Technology	Role in the solution
Orchestration	LangGraph	Defines the 5-node StateGraph pipeline; streams node updates to the UI.
LLM framework	LangChain (core, groq, openai)	Provider abstraction; wraps Groq (primary) and Azure OpenAI (fallback); prompt/message plumbing.
Structured AI + RAG	LlamaIndex	structured_predict for JD parsing; VectorStoreIndex + SentenceSplitter + metadata filters for retrieval.
LLM (primary)	Groq — llama-3.3-70b-versatile	Fast, low-cost inference for parsing, scoring and gap analysis (temperature 0).
LLM (fallback)	Azure OpenAI / OpenAI — gpt-4o-mini	Automatic failover when Groq is rate-limited.
Embeddings	HuggingFace all-MiniLM-L6-v2	Local CPU sentence embeddings (~80MB, cached) — no API key, no cost.
Vector store	Chroma (persistent, local)	Stores profile chunk vectors in ./chroma_db (collection 'profiles_li').
Job data	Exa (exa-py) + optional Exa MCP	Neural search & full-content fetch of job postings; MCP variant via langchain-mcp-adapters.
Web UI	Streamlit	Interactive app: input modes, live progress, metrics, tables, downloads.
Reporting	ReportLab	Generates the cumulative ATS-Report.pdf from the JSON run log.
Observability	LangSmith (optional)	Tracing of LLM/graph runs when LANGCHAIN_TRACING_V2 is enabled.

Layer	Technology	Role in the solution
Data models	Pydantic v2	Typed _ParsedJD schema enforced during structured extraction.
Config / secrets	python-dotenv (.env)	Loads API keys and paths; keeps secrets out of source.
Proxy handling	ssl_patch.py	Points REQUESTS_CA_BUNDLE / SSL_CERT_FILE at a corporate proxy CA bundle.

4 How AI Is Plugged In

Where the LLM is called

The LLM is used at three distinct stages of the pipeline, each with a purpose-built prompt and deterministic settings (`temperature = 0`) for reproducible results:

- **JD parsing** — LlamaIndex `structured_predict` maps free JD text onto the `_ParsedJD` Pydantic schema.
- **Profile scoring** — a strict ATS prompt returns a JSON object with IC/Leadership scores, strengths, gaps and the recommended profile.
- **Gap analysis** — a coaching prompt turns gaps into grounded, actionable levers.

Multi-provider resilience

A single helper, `_invoke_llm(messages)`, calls Groq first and transparently fails over to Azure OpenAI / OpenAI when a rate-limit signal is detected (matching on `rate_limit, 429, quota, too many requests`). LLM clients are lazily instantiated and cached as singletons.

```
try:  return get_llm().invoke(messages)  # Groq
except Exception as e:  # on rate-limit → Azure/OpenAI fallback
    if any(sig in str(e).lower() for sig in RATE_LIMIT_SIGNALS):
        return get_fallback_llm().invoke(messages)
```

Robustness patterns

- **Schema-first extraction:** structured_predict reduces malformed-JSON risk; a LangChain JSON path with regex extraction is the fallback if LlamaIndex/Groq errors.
- **Deterministic scoring:** temperature 0 keeps scores stable across identical inputs.
- **Graceful degradation:** every stage has a fallback so one provider or parser failure does not break the run.

5 How RAG (Retrieval-Augmented Generation) Is Used

RAG is the core of the scoring stage. Rather than pasting entire multi-page profiles into the prompt (costly and noisy), the system retrieves only the profile passages most relevant to each job's requirements and grounds the LLM's score in that evidence.

Ingestion pipeline (LlamaIndex → Chroma)

Step	Detail
1. Load	IC and Leadership profile .txt files become LlamaIndex Documents, tagged with metadata profile_type = 'IC' | 'Leadership'.

Step	Detail
2. Chunk	SentenceSplitter(chunk_size=512, chunk_overlap=64) splits each profile into semantically coherent nodes (~12 chunks from 2 profiles).
3. Embed	Each chunk is embedded locally with HuggingFace all-MiniLM-L6-v2 (CPU, normalized).
4. Persist	Vectors + metadata are written to a local Chroma collection ('profiles_li') via ChromaVectorStore, persisted under ./chroma_db.

Retrieval & augmentation

- For each JD, the parsed requirements + nice-to-haves are concatenated into a query string.
- A **metadata-filtered retriever** fetches the top-**k=3** chunks for **IC** and, separately, the top-3 for **Leadership** (MetadataFilters on profile_type).
- The retrieved chunks are injected into the scoring prompt, so the LLM scores each persona against real, targeted evidence — not the whole profile and not hallucinated experience.
- `refresh_profiles()` re-ingests (reset=True) after profile edits; the index is otherwise cached and reused without re-embedding.

Why RAG fits this problem

- **Two personas, one query:** metadata filtering cleanly separates IC vs Leadership evidence from a shared store.
- **Context efficiency:** only the most relevant passages enter the prompt, improving keyword-level scoring accuracy and controlling token usage.
- **Local & private:** embeddings and vectors never leave the machine — no embedding API cost and no data egress.

6 Pros & Cons of the Tools Chosen

Tool	Pros	Cons / trade-offs
LangGraph	Explicit typed state; inspectable, testable nodes; easy to add/remove stages; native streaming.	Extra abstraction for a currently linear flow; another dependency to learn.
LangChain	Uniform LLM interface; trivial provider swap/fallback; rich ecosystem.	Fast-moving API surface; heavier dependency; occasional breaking changes.
LlamaIndex	First-class structured extraction; ergonomic RAG ingestion & metadata filtering.	Overlaps LangChain in scope; running both adds cognitive load and deps.
Groq (Llama 3.3)	Very fast and cheap; strong 70B quality for parsing/scoring.	Tight rate limits on free tier (drove the fallback design).
Azure OpenAI (fallback)	Reliable, enterprise-grade, higher limits; smooth failover target.	Paid; requires endpoint/key setup; adds a second provider to manage.
HuggingFace MiniLM	Local, free, private, no API key; good quality for short corpora.	CPU embedding is slower at scale; ~80MB first-time model download.
Chroma	Zero-config, local, persistent; ideal for a small corpus.	Not built for very large / multi-tenant / distributed workloads.
Exa	Neural search + clean full-content fetch of postings via one API.	Index coverage of gated boards (LinkedIn/Glassdoor) is sparse & fuzzy.

Tool	Pros	Cons / trade-offs
MCP (Exa server)	Standardised tool protocol; portable across agents; resume-relevant.	Needs Node.js/npx; less dependable than the SDK — SDK kept as fallback.
Streamlit	Very fast to build data UIs; live progress; minimal front-end code.	Single-user session model; limited layout control; not for high-concurrency.
ReportLab	Precise, dependency-light PDF generation entirely in Python.	Verbose imperative layout API compared with HTML-to-PDF tools.

7 Learnings — Evolution From Start to End

What changed over the build

Area	Started as	Ended as	Why
Retrieval stack	LangChain-only Chroma ('profiles' collection)	LlamaIndex ingestion + retrieval ('profiles_li')	Structured extraction + metadata-filtered RAG; avoid schema clash.
JD parsing	LLM + hand-parsed JSON	LlamaIndex structured_predict (+ JSON fallback)	Schema-enforced output; fewer parse failures.
LLM reliability	Single provider (Groq)	Groq → Azure/OpenAI auto-fallback	Free-tier rate limits interrupted runs.
Job sourcing	Search Query, LinkedIn URL, Glassdoor crawl (+cookie)	Job URL, Job URLs file, Paste JD text	Scraping/search returned wrong or no jobs; simplified to reliable inputs.
Cover letter	LLM cover-letter node + UI	Removed	Out of scope for scoring; reduced surface area and cost.

Key engineering lessons

- **Prefer reliable inputs over brittle scraping.** LinkedIn blocks crawlers and Glassdoor's 'Jobs for you' feed renders client-side; Exa's index of these is fuzzy. Deterministic URL/paste inputs produce far better results than automated search.
- **Design for provider limits up front.** A thin fallback wrapper (_invoke_llm) turned rate-limit failures from run-killers into transparent retries.
- **Use the right framework per job.** LangChain for provider abstraction; LlamaIndex for structured extraction and RAG — combined rather than forcing one to do everything.
- **Isolate cross-cutting concerns.** Secrets in .env; corporate-proxy TLS in ssl_patch.py imported first; a separate collection name to prevent vector-schema conflicts.
- **Keep the graph lean.** Removing unused modes/nodes reduced cost, failure points and cognitive load without losing core value.
- **Ground the model.** Metadata-filtered RAG + temperature 0 makes scoring more honest, repeatable and defensible than free-form prompting.

8 FAQ (Anticipated Assessment Questions)

Q. Why LangGraph instead of a plain script or a LangChain chain?

A. The workflow is a series of stateful stages that must be inspectable, individually testable and streamable to the UI. LangGraph gives a typed shared state and explicit nodes/edges, so stages can be added, removed or reordered safely — which is exactly what happened when the cover-letter node was removed.

Q. Why use both LangChain and LlamaIndex — isn't that redundant?

A. They are used for complementary strengths. LangChain provides the LLM provider abstraction and the Groq→Azure fallback; LlamaIndex provides schema-enforced structured extraction and an ergonomic, metadata-filtered RAG pipeline. Each does what it is best at.

Q. Why local embeddings and Chroma rather than a cloud vector database?

A. The corpus is tiny (two profiles) and private. Local MiniLM embeddings + local Chroma mean zero API cost, no data egress, and offline operation. A cloud store would add cost and complexity with no benefit at this scale.

Q. How do you stop the LLM from hallucinating scores?

A. Three controls: (1) RAG grounds scoring in retrieved, real profile passages; (2) temperature 0 makes output deterministic; (3) a strict prompt demands honest gaps and a fixed JSON schema. Scoring is still model-assisted judgement, so it is decision-support, not an absolute measure — a known limitation.

Q. How is rate limiting handled?

A. _invoke_llm calls Groq first and, on any rate-limit signal (rate_limit/429/quota/too many requests), transparently retries on Azure OpenAI / OpenAI. Bulk scoring also spaces calls to stay within limits.

Q. Why were the search / LinkedIn / Glassdoor modes removed?

A. They did not reliably return the right jobs: LinkedIn blocks automated crawling, Glassdoor's personalised feed is client-side rendered, and Exa's index of those boards is sparse and fuzzy. The app was simplified to deterministic inputs (Job URL, URL file, pasted JD) that consistently work.

Q. How is the ATS report persisted and kept consistent?

A. Each run appends a record to output/ats_log.json, then the full ATS-Report.pdf is rebuilt from the log with ReportLab. Rebuilding from the source of truth guarantees the PDF always reflects every run.

Q. How are secrets and corporate-proxy TLS handled?

A. All keys and paths load from .env via python-dotenv (never hard-coded). ssl_patch.py is imported first in both entry points to point the TLS trust store at an optional corporate proxy CA bundle so model downloads and HTTPS calls succeed behind an inspecting proxy.

Q. What are the main limitations today?

A. It is single-user; scoring is sequential (with deliberate pauses for rate limits); job sourcing relies on user-supplied URLs; and scores reflect LLM judgement. These are acceptable for a personal tool but would need work for production.

Q. How would you productionise this?

A. Add async/batch scoring and caching; a provider pool with health-based routing; a proper evaluation harness (e.g. LangSmith datasets) to measure scoring quality; authentication and multi-user persistence; and a managed vector store if the corpus grows beyond a handful of profiles.

Q. What is the role of MCP here?

A. Exa is integrated through the Model Context Protocol (via langchain-mcp-adapters) as a standardised, portable tool interface, with the direct Exa SDK kept as a dependable fallback when Node.js/MCP is unavailable.

Generated from the live codebase for technical-assessment purposes. Pipeline: fetch_jd → parse_jd → score_profiles → analyse_gaps → update_ats.